DATA STRUCTURE MOCK EXAM 2025

Python Programs: Questions 1 to 19

Programs are written in simple Python. For list inputs, enter values separated by spaces. Graph vertices are numbered from 0 to n-1.

1. Library Management: Average, Highest/Lowest, Zero Borrowings and Mode

```
n = int(input("Enter number of members: "))
names = []
books = []

for i in range(n):
    name = input("Enter member name: ")
    count = int(input("Enter books borrowed: "))
    names.append(name)
    books.append(count)

if n == 0:
    print("No members")
else:
    total = sum(books)
    print("Average:", total / n)

    high = max(books)
    low = min(books)
    print("Highest borrower:", names[books.index(high)], high)
    print("Lowest borrower:", names[books.index(low)], low)
    print("Members with zero books:", books.count(0))

    freq = {}
    for count in books:
        freq[count] = freq.get(count, 0) + 1

    highest_freq = max(freq.values())
    modes = [k for k, v in freq.items() if v == highest_freq]
    print("Most frequent borrow count:", modes)
```

2. Student Marks: Average, Highest/Lowest, Absent Count and Mode

```
n = int(input("Enter number of students: "))
marks = []

for i in range(n):
    mark = int(input("Enter marks (-1 for absent): "))
    marks.append(mark)

present = [m for m in marks if m != -1]
absent = marks.count(-1)

print("Absent students:", absent)

if present:
    print("Average:", sum(present) / len(present))
    print("Highest marks:", max(present))
    print("Lowest marks:", min(present))

    freq = {}
    for m in present:
        freq[m] = freq.get(m, 0) + 1
    max_freq = max(freq.values())
    print("Most frequent marks:",
          [m for m, c in freq.items() if c == max_freq])
else:
    print("No students attended the test")
```

3. Linear Search: Customer Account IDs

```
ids = list(map(int, input("Enter account IDs: ").split()))
key = int(input("Enter ID to search: "))

found = False
for i in range(len(ids)):
    if ids[i] == key:
        print("Account ID found at position", i + 1)
        found = True
        break
```

```
if not found:
    print("Account ID not found")
```

4. Binary Search: Customer Account IDs

```
ids = list(map(int, input("Enter sorted account IDs: ").split()))
key = int(input("Enter ID to search: "))

low = 0
high = len(ids) - 1
found = False

while low <= high:
    mid = (low + high) // 2

    if ids[mid] == key:
        print("Account ID found at position", mid + 1)
        found = True
        break
    elif key < ids[mid]:
        high = mid - 1
    else:
        low = mid + 1

if not found:
    print("Account ID not found")
```

5. Linear Search: Training Attendance Roll Numbers

```
rolls = list(map(int, input("Enter roll numbers in any order: ").split()))
key = int(input("Enter roll number to search: "))

found = False
for roll in rolls:
    if roll == key:
        found = True
        break

if found:
    print("Student attended the training")
else:
    print("Student did not attend the training")
```

6. Binary Search: Training Attendance Roll Numbers

```
rolls = list(map(int, input("Enter roll numbers in ascending order: ").split()))
key = int(input("Enter roll number to search: "))

low = 0
high = len(rolls) - 1
found = False

while low <= high:
    mid = (low + high) // 2
    if rolls[mid] == key:
        found = True
        break
    elif key < rolls[mid]:
        high = mid - 1
    else:
        low = mid + 1

print("Student attended the training" if found
      else "Student did not attend the training")
```

7. Employee Salaries: Bubble Sort and Top Five

```
salary = list(map(float, input("Enter salaries: ").split()))

n = len(salary)
for i in range(n - 1):
    for j in range(n - i - 1):
        if salary[j] > salary[j + 1]:
            salary[j], salary[j + 1] = salary[j + 1], salary[j]

print("Ascending salaries:", salary)
print("Top five highest salaries:", salary[-5:][::-1])
```

8. Student Percentages: Selection Sort and Top Five

```
marks = list(map(float, input("Enter percentages: ").split()))

n = len(marks)
for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if marks[j] < marks[min_index]:
            min_index = j
    marks[i], marks[min_index] = marks[min_index], marks[i]

print("Ascending percentages:", marks)
print("Top five scores:", marks[-5:][::-1])
```

9. Student Percentages: Bubble Sort and Top Five

```
marks = list(map(float, input("Enter percentages: ").split()))

n = len(marks)
for i in range(n - 1):
    for j in range(n - i - 1):
        if marks[j] > marks[j + 1]:
            marks[j], marks[j + 1] = marks[j + 1], marks[j]

print("Ascending percentages:", marks)
print("Top five scores:", marks[-5:][::-1])
```

10. Undo/Redo System Using Two Stacks

```
document = ""
undo_stack = []
redo_stack = []

while True:
    print("\n1. Make change  2. Undo  3. Redo  4. Display  5. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        undo_stack.append(document)
        redo_stack.clear()
        document = input("Enter new document text: ")
    elif choice == "2":
        if undo_stack:
            redo_stack.append(document)
            document = undo_stack.pop()
        else:
            print("Nothing to undo")
    elif choice == "3":
        if redo_stack:
            undo_stack.append(document)
            document = redo_stack.pop()
        else:
            print("Nothing to redo")
    elif choice == "4":
        print("Document:", document)
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

11. Event Processing System Using Queue

```
queue = []

while True:
    print("\n1. Add event  2. Process next  3. Display  4. Cancel  5. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        event = input("Enter event: ")
        queue.append(event)
        print("Event added")
    elif choice == "2":
        if queue:
            print("Processed:", queue.pop(0))
        else:
            print("No pending events")
    elif choice == "3":
        print("Pending events:", queue)
    elif choice == "4":
        event = input("Enter event to cancel: ")
        if event in queue:
            queue.remove(event)
```

```
            print("Event cancelled")
        else:
            print("Event not found")
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

12. Call Center Queue

```
calls = []

def addCall(customerID, callTime):
    calls.append((customerID, callTime))
    print("Call added")

def answerCall():
    if calls:
        print("Answered call:", calls.pop(0))
    else:
        print("No calls waiting")

def viewQueue():
    print("Waiting calls:", calls)

def isQueueEmpty():
    return len(calls) == 0

while True:
    print("\n1.Add call  2.Answer call  3.View queue  4.Check empty  5.Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        cid = input("Customer ID: ")
        time = int(input("Call time in minutes: "))
        addCall(cid, time)
    elif choice == "2":
        answerCall()
    elif choice == "3":
        viewQueue()
    elif choice == "4":
        print("Queue empty:", isQueueEmpty())
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

13. Job Queue Using Queue

```
jobs = []

def add_job(job):
    jobs.append(job)
    print("Job added")

def delete_job():
    if jobs:
        print("Processed job:", jobs.pop(0))
    else:
        print("Job queue is empty")

while True:
    print("\n1. Add job  2. Delete/process job  3. Display  4. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        add_job(input("Enter job name: "))
    elif choice == "2":
        delete_job()
    elif choice == "3":
        print("Job queue:", jobs)
    elif choice == "4":
        break
    else:
        print("Invalid choice")
```

14. Hash Table of Size 10 Using Chaining

```
SIZE = 10
table = [[] for _ in range(SIZE)]

def hash_function(key):
```

```
    return key % SIZE

def insert(key, value):
    index = hash_function(key)
    for i, pair in enumerate(table[index]):
        if pair[0] == key:
            table[index][i] = (key, value)
            return
    table[index].append((key, value))

def search(key):
    index = hash_function(key)
    for k, value in table[index]:
        if k == key:
            return value
    return None

def delete(key):
    index = hash_function(key)
    for pair in table[index]:
        if pair[0] == key:
            table[index].remove(pair)
            print("Key deleted")
            return
    print("Key not found")

while True:
    print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        key = int(input("Key: "))
        value = input("Value: ")
        insert(key, value)
    elif choice == "2":
        key = int(input("Key: "))
        print("Value:", search(key))
    elif choice == "3":
        delete(int(input("Key: ")))
    elif choice == "4":
        print(table)
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

15. Hash Table Using Linear Probing

```
SIZE = 10
table = [None] * SIZE

def insert(key):
    index = key % SIZE
    for i in range(SIZE):
        pos = (index + i) % SIZE
        if table[pos] is None or table[pos] == key:
            table[pos] = key
            print("Key inserted")
            return
    print("Hash table is full")

def search(key):
    index = key % SIZE
    for i in range(SIZE):
        pos = (index + i) % SIZE
        if table[pos] is None:
            return False
        if table[pos] == key:
            return True
    return False

def delete(key):
    index = key % SIZE
    for i in range(SIZE):
        pos = (index + i) % SIZE
        if table[pos] is None:
            break
        if table[pos] == key:
            table[pos] = "DELETED"
            print("Key deleted")
            return
    print("Key not found")
while True:
```

```
    print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        insert(int(input("Key: ")))
    elif choice == "2":
        print("Found" if search(int(input("Key: "))) else "Not found")
    elif choice == "3":
        delete(int(input("Key: ")))
    elif choice == "4":
        print(table)
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

16. BFS Using Adjacency List

```
n = int(input("Enter number of vertices: "))
adj = [[] for _ in range(n)]

e = int(input("Enter number of edges: "))
for i in range(e):
    u, v = map(int, input("Enter edge (u v): ").split())
    adj[u].append(v)
    adj[v].append(u)

start = int(input("Enter starting vertex: "))
visited = [False] * n
queue = [start]
visited[start] = True

print("BFS:", end=" ")
while queue:
    node = queue.pop(0)
    print(node, end=" ")

    for neighbour in adj[node]:
        if not visited[neighbour]:
            visited[neighbour] = True
            queue.append(neighbour)
```

17. DFS Using Adjacency Matrix

```
n = int(input("Enter number of vertices: "))
matrix = [[0] * n for _ in range(n)]

e = int(input("Enter number of edges: "))
for i in range(e):
    u, v = map(int, input("Enter edge (u v): ").split())
    matrix[u][v] = 1
    matrix[v][u] = 1

start = int(input("Enter starting vertex: "))
visited = [False] * n
stack = [start]

print("DFS:", end=" ")
while stack:
    node = stack.pop()

    if not visited[node]:
        visited[node] = True
        print(node, end=" ")

        for i in range(n - 1, -1, -1):
            if matrix[node][i] == 1 and not visited[i]:
                stack.append(i)
```

18. Binary Search Tree: Insert, Search, Delete and Display

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
```

```
    elif key > root.key:
        root.right = insert(root.right, key)
    return root

def search(root, key):
    if root is None or root.key == key:
        return root
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)

def minimum(root):
    while root.left:
        root = root.left
    return root

def delete(root, key):
    if root is None:
        return None
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        temp = minimum(root.right)
        root.key = temp.key
        root.right = delete(root.right, temp.key)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

root = None
while True:
    print("\n1.Insert  2.Search  3.Delete  4.Display  5.Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        root = insert(root, int(input("Key: ")))
    elif choice == "2":
        key = int(input("Key: "))
        print("Found" if search(root, key) else "Not found")
    elif choice == "3":
        root = delete(root, int(input("Key: ")))
    elif choice == "4":
        print("Inorder:", end=" ")
        inorder(root)
        print()
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

19. Expression Tree: Prefix, Non-recursive Post-order and Delete

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def build_tree(prefix):
    stack = []

    for ch in reversed(prefix):
        if ch.isalnum():
            stack.append(Node(ch))
        else:
            if len(stack) < 2:
                raise ValueError("Invalid prefix expression")
            node = Node(ch)
            node.left = stack.pop()
            node.right = stack.pop()
            stack.append(node)

    if len(stack) != 1:
```

```
            raise ValueError("Invalid prefix expression")
        return stack.pop()

    def postorder_non_recursive(root):
        if root is None:
            return

        stack1 = [root]
        stack2 = []

        while stack1:
            node = stack1.pop()
            stack2.append(node)
            if node.left:
                stack1.append(node.left)
            if node.right:
                stack1.append(node.right)

        while stack2:
            print(stack2.pop().data, end=" ")

    def delete_tree(root):
        if root is None:
            return

        stack = [root]
        while stack:
            node = stack.pop()
            if node.left:
                stack.append(node.left)
            if node.right:
                stack.append(node.right)
            node.left = None
            node.right = None
            node.data = None

        print("\nTree deleted successfully")

    prefix = input("Enter prefix expression: ")
    try:
        root = build_tree(prefix)
        print("Post-order traversal:")
        postorder_non_recursive(root)
        delete_tree(root)
    except ValueError as e:
        print(e)
```